

PROJECT_01

작심하루

배포 링크

Github 링크

테스트 ID - test12@a.com

테스트 PW - test1234!

개발 기간

2024년 8월 26일 - 2024년 9월 24일

개발 인원

총 4인(프론트엔드 4)

담당 역할

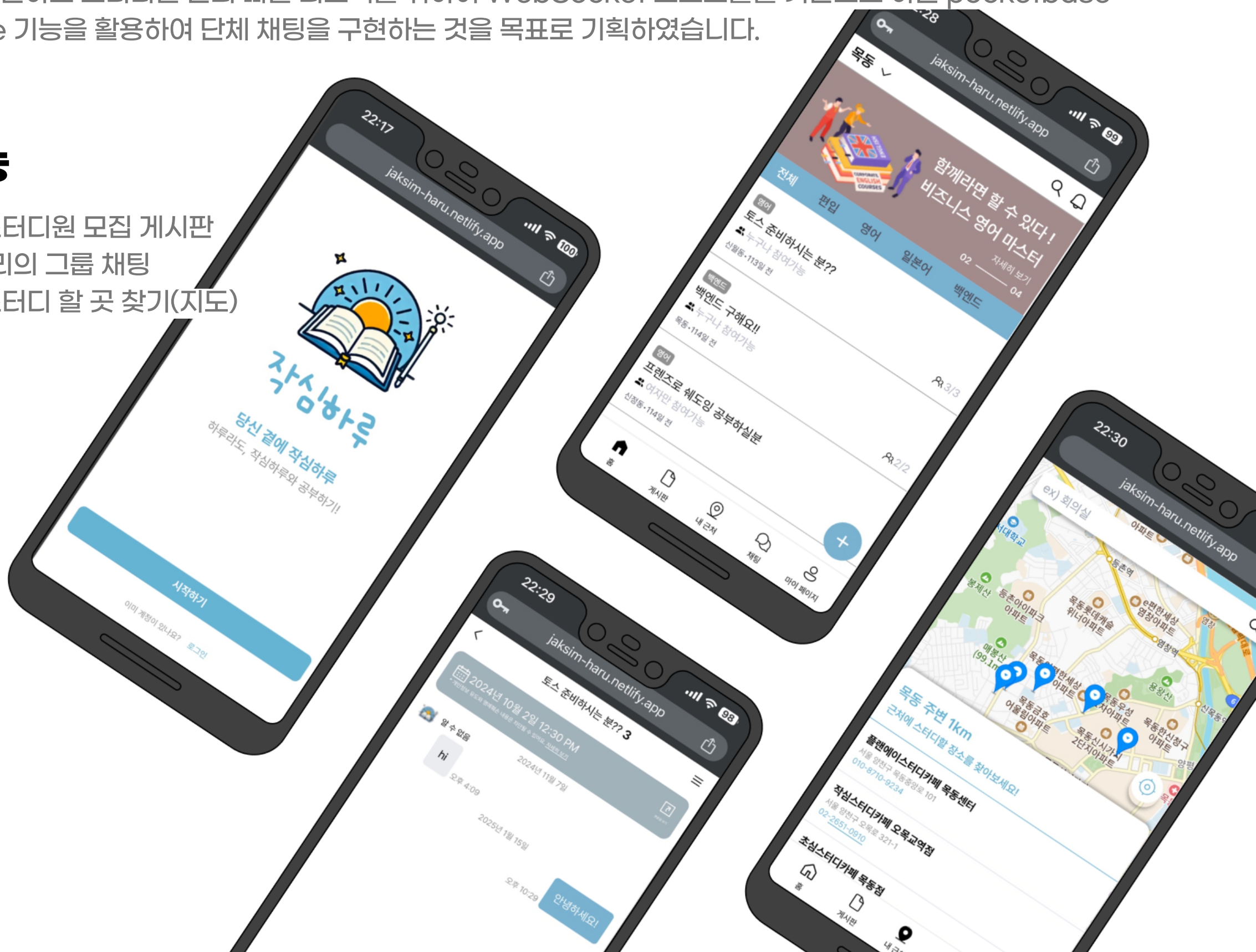
조장(프로젝트 총괄)

기획 의도

"작심하루"는 내가 원하는 지역에서 빠르게 스터디원을 구하는 모바일 전용 플랫폼입니다. 빠르고 쉽게 스터디를 만들거나 가입할 수 있도록 하기 위하여 기획되었습니다. 사용자 위치 근처 지도를 확인할 수 있도록 kakao map api를 통한 지도를 구현하고 스터디원 간의 빠른 피드백을 위하여 WebSocket 프로토콜을 기반으로 하는 pocketbase의 subscribe 기능을 활용하여 단체 채팅을 구현하는 것을 목표로 기획하였습니다.

주요 기능

- 우리 동네 스터디원 모집 게시판
- 스터디원끼리의 그룹 채팅
- 우리 동네 스터디 할 곳 찾기(지도)



PROJECT_01

작심하루

 배포 링크

 Github 링크

테스트 ID - test12@a.com

테스트 PW - test1234!

개발 기간

2024년 8월 26일 - 2024년 9월 24일

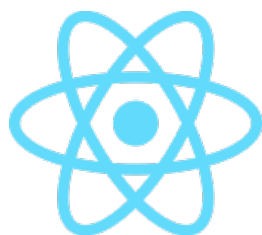
개발 인원

총 4인(프론트엔드 4)

담당 역할

조장(프로젝트 총괄)

사용 기술



React

아토믹 컴포넌트를 제작하여 재사용이 가능하도록 하고 SPA를 통해 유연한 UI를 제공하기 위해 선택



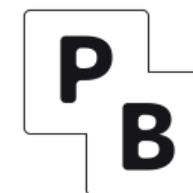
TailwindCSS

클래스 기반의 스타일링으로 CSS를 따로 작성할 필요가 없으며 반응형 디자인에 유리하여 선택



Zustand

전역에서 관리해야하는 상태관리가 필요했고, 상태관리 라이브러리 중에서도 간편하고 가벼운 라이브러리기 때문에 선택



Pocketbase

프론트엔드끼리만 하는 프로젝트였기 때문에 간편하게 DB 관리할 수 있는 포켓베이스를 선택



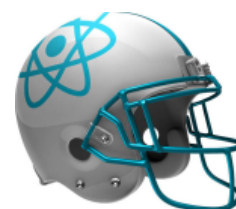
React-Router-Dom

SPA에서 동적 페이지 전환이 가능하고 navigate와 같은 웹 전용 기능을 사용하기 위해 선택



GSAP

CSS보다 부드럽고 timeline과 같은 순차적 애니메이션 기능이 적용 가능하여 선택



React-Helmet

React의 경우 메타 태그 설정이 어렵기 때문에 SEO와 메타 태그 관리를 위하여 사용



Vite

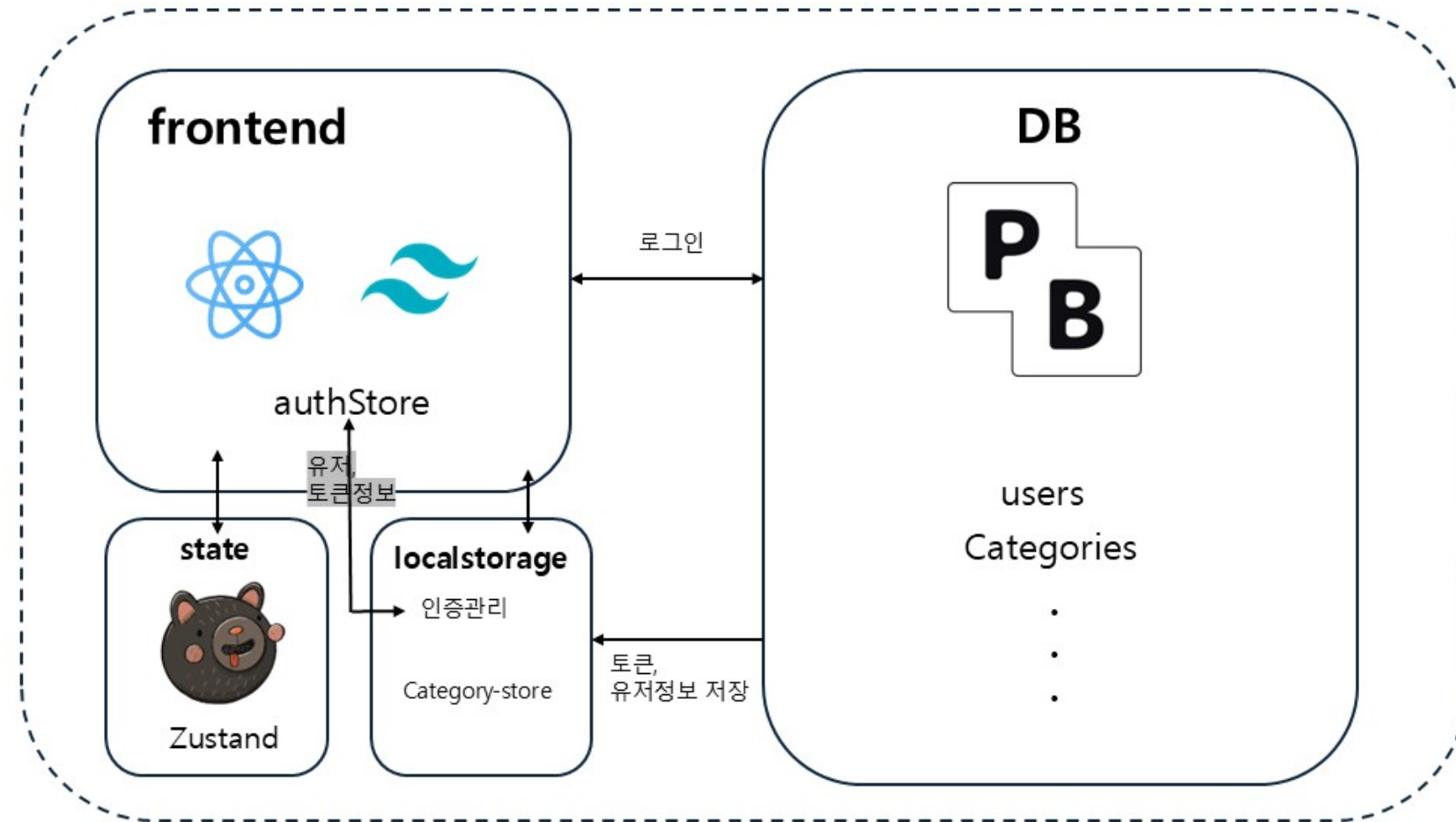
빠른 빌드 속도와 HMR을 지원하여 변경된 모듈만 확인할 수 있다는 장점으로 선택



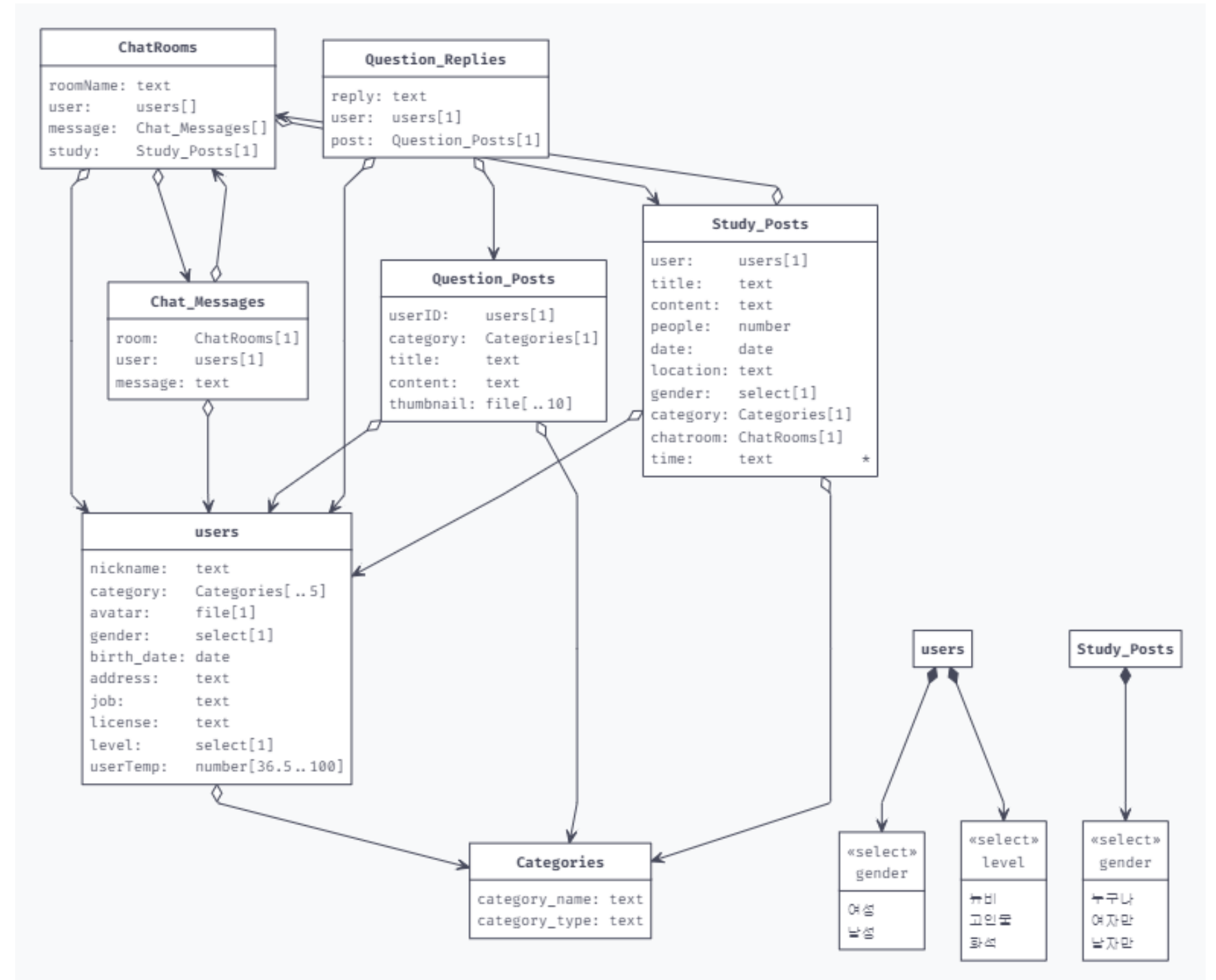
Netlify

이전 프로젝트에서도 사용해본 경험이 있고 무료로 쉽게 배포할 수 있기 때문에 선택

아키텍처



ERD



수행 업무 및 기여(%)

채팅 페이지 100%

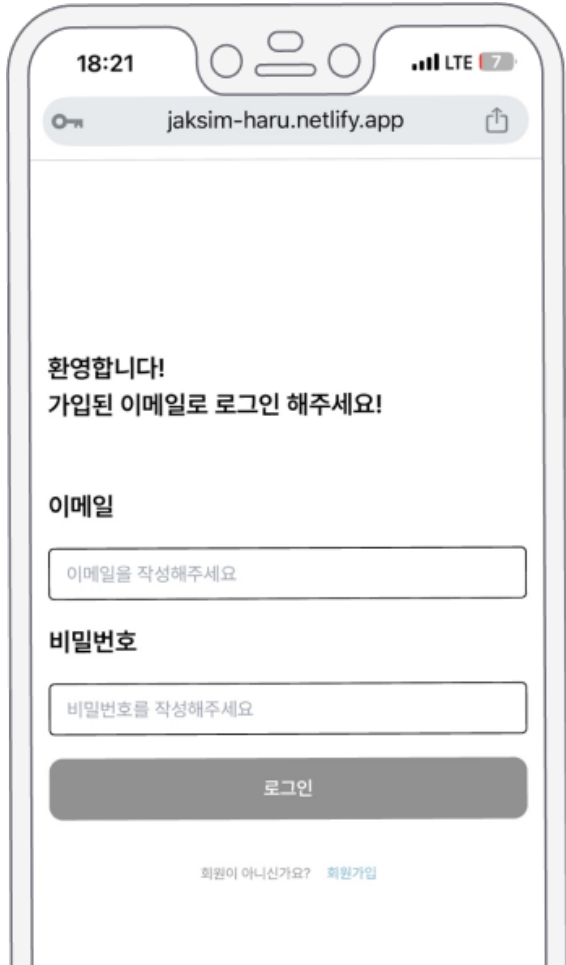
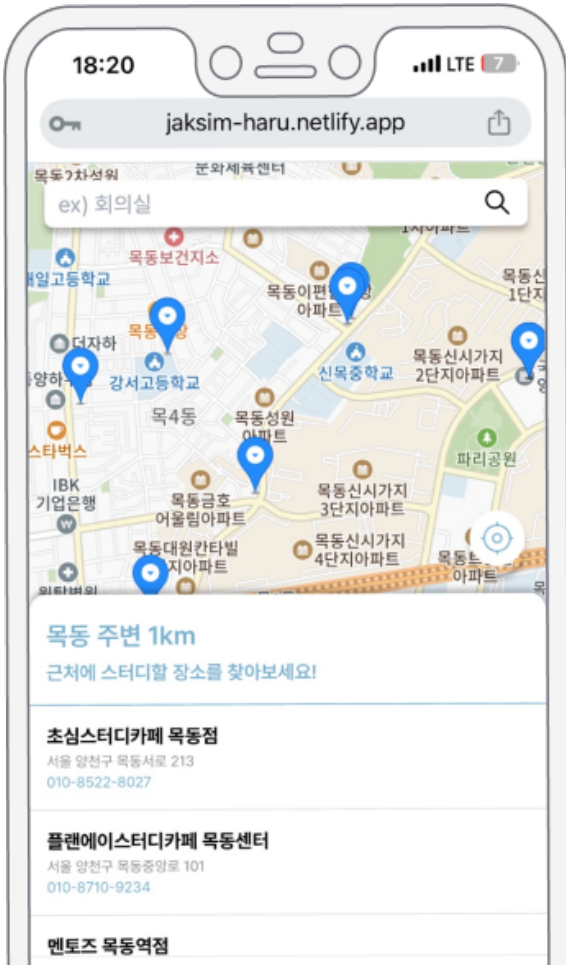
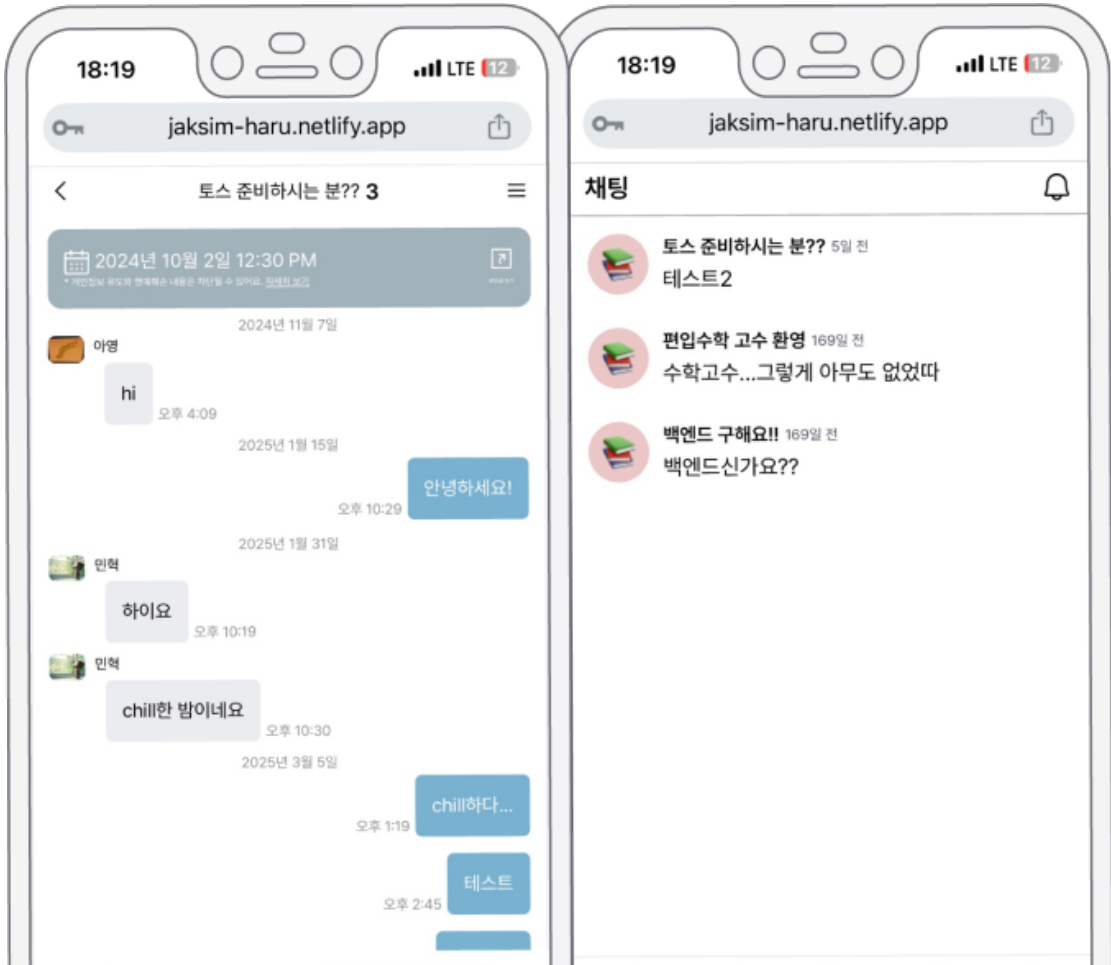
- 프론트엔드
- 사용자가 입장한 채팅방 리스트 렌더링
- WebSocket 프로토콜을 기반으로 하는 pocketbase의 subscribe를 활용한 단체 채팅 기능
- 채팅방 나가기, 유저정보 확인 기능
- 채팅 작성 시 스크롤 자동 내려가기
- 채팅방 생성시 해당 스터디원 모집 게시글 정보 불러오기

내 근처 페이지 100%

- 프론트엔드
- 카카오맵 API를 통한 사용자 설정 위치 정보에 따른 근처 공부할만한 곳 추천 리스트 렌더링
- 검색 기능을 통한 장소 검색 기능

로그인 페이지 100%

- 프론트엔드
- 로그인 유효성 검사
- 로그인 조건 만족 시 로그인 버튼 활성화(default: 비활성화)
- 로그인 기능



수행 업무 및 기여(%)

질문 게시글 작성 페이지 100%

- 프론트엔드
- 이미지, 제목, 텍스트, 관심분야 게시 기능

질문 상세 페이지, 수정 100%

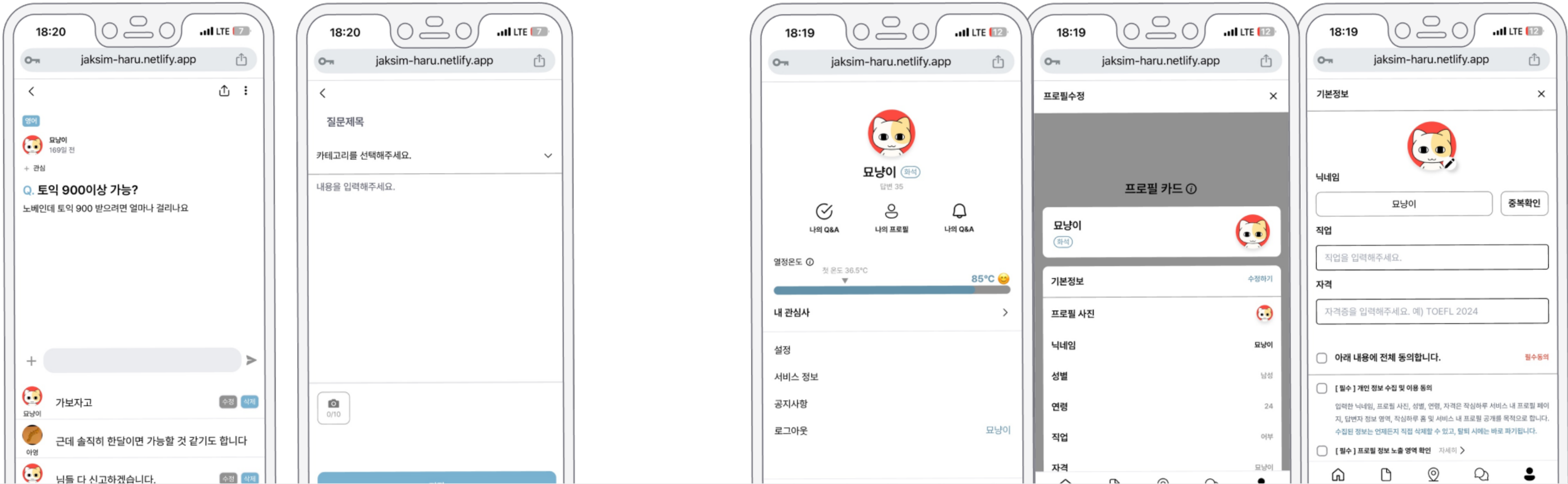
- 프론트엔드
- 게시글 삭제, 수정
- 댓글 기능(게시, 삭제, 수정)

마이 페이지 100%

- 프론트엔드
- 프로필 정보 불러오기
- 로그아웃 기능 구현
- 프로필 정보 수정 구현

ETC

- 조장 - 데일리 스크럼, 스프린트 회고 진행
- github wiki, README.md 작성
- 팀 컨벤션 및 규칙 정리 등 프로젝트 총괄
- DB 설계
- 회원가입 시 SMTP를 활용한 Email 인증 메일 전송



트러블 슈팅

문제 정의

홈페이지 접근 -> 게시판 페이지 접근 (정상적인 렌더링)
다른 페이지 -> 게시판 페이지 (정상적인 렌더링 X)
새로고침 -> 정상적인 렌더링 X

원인

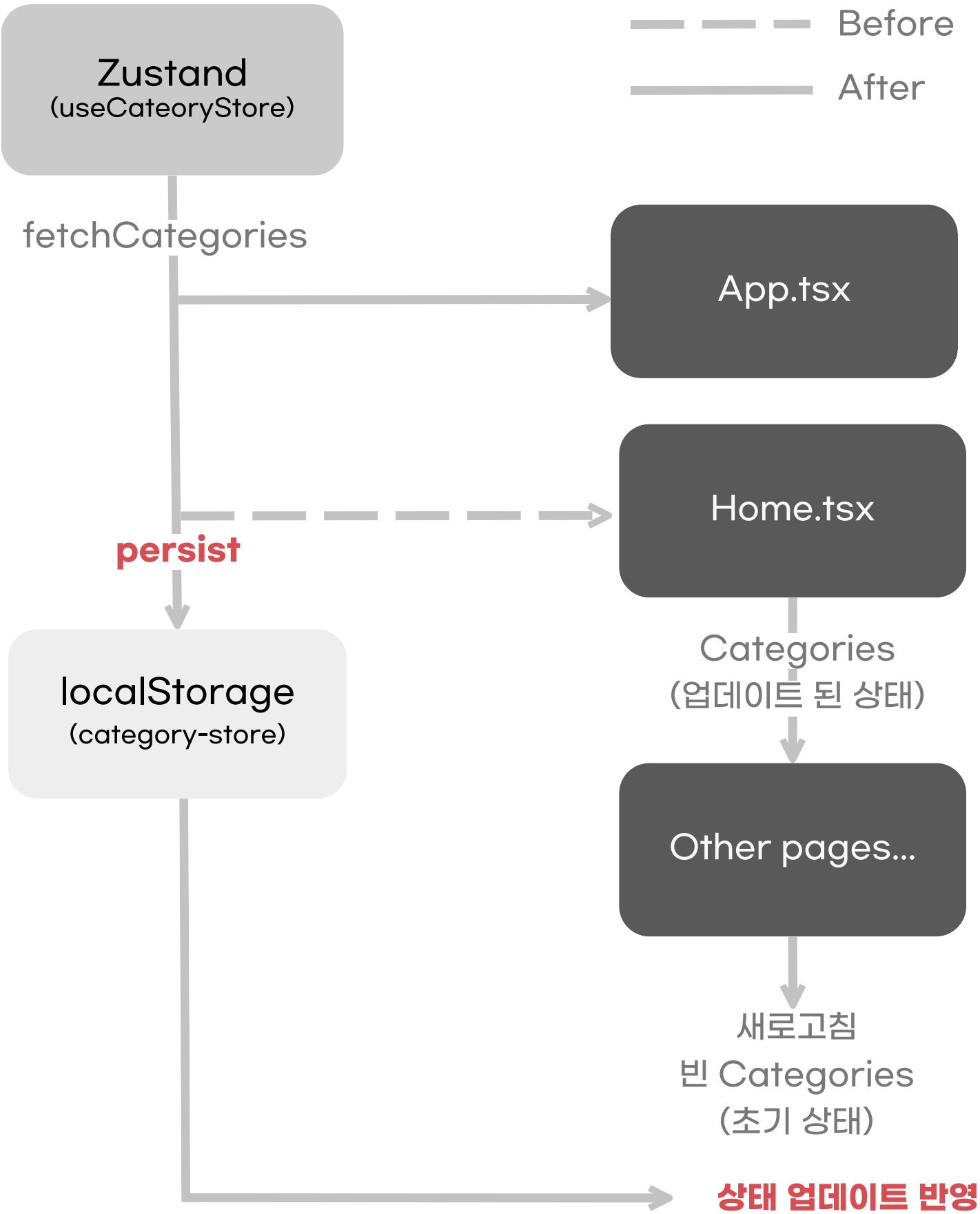
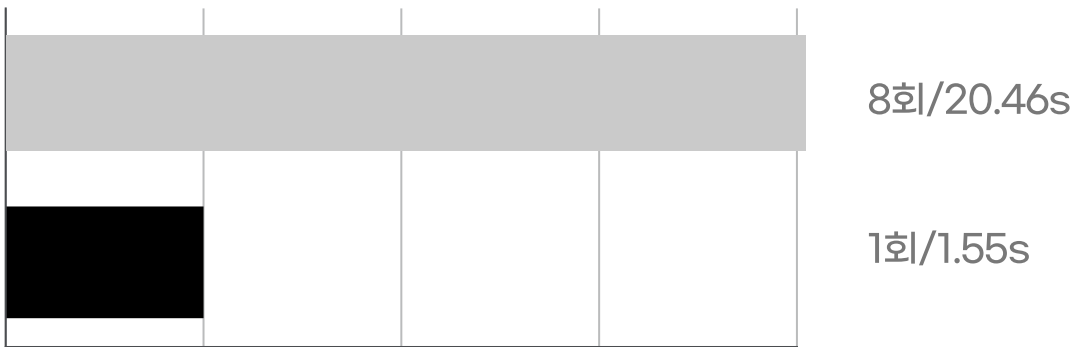
store로 관리되던 categories 상태의 경우 state가 새로고침하면 초기화 되기 때문에 카테고리 데이터가 초기화 되어 게시판 페이지의 게시글 렌더링이 정상적으로 이루어지지 않는 문제였다.

해결방법

- 글로벌 페이지(App.tsx)에서 상태 업데이트 되도록 수정
- 잦은 네트워크 요청 감소를 위해 **zustand persist 적용하여** 캐싱

기대 효과

- 요청 시간 감소
- 요청 횟수 감소



트러블 슈팅

문제 정의

댓글 렌더링을 하는 과정에서 user 데이터를 expand를 하는 경우 user를 reading 할 수 없다는 type error가 발생

원인

클라이언트에서 댓글 작성

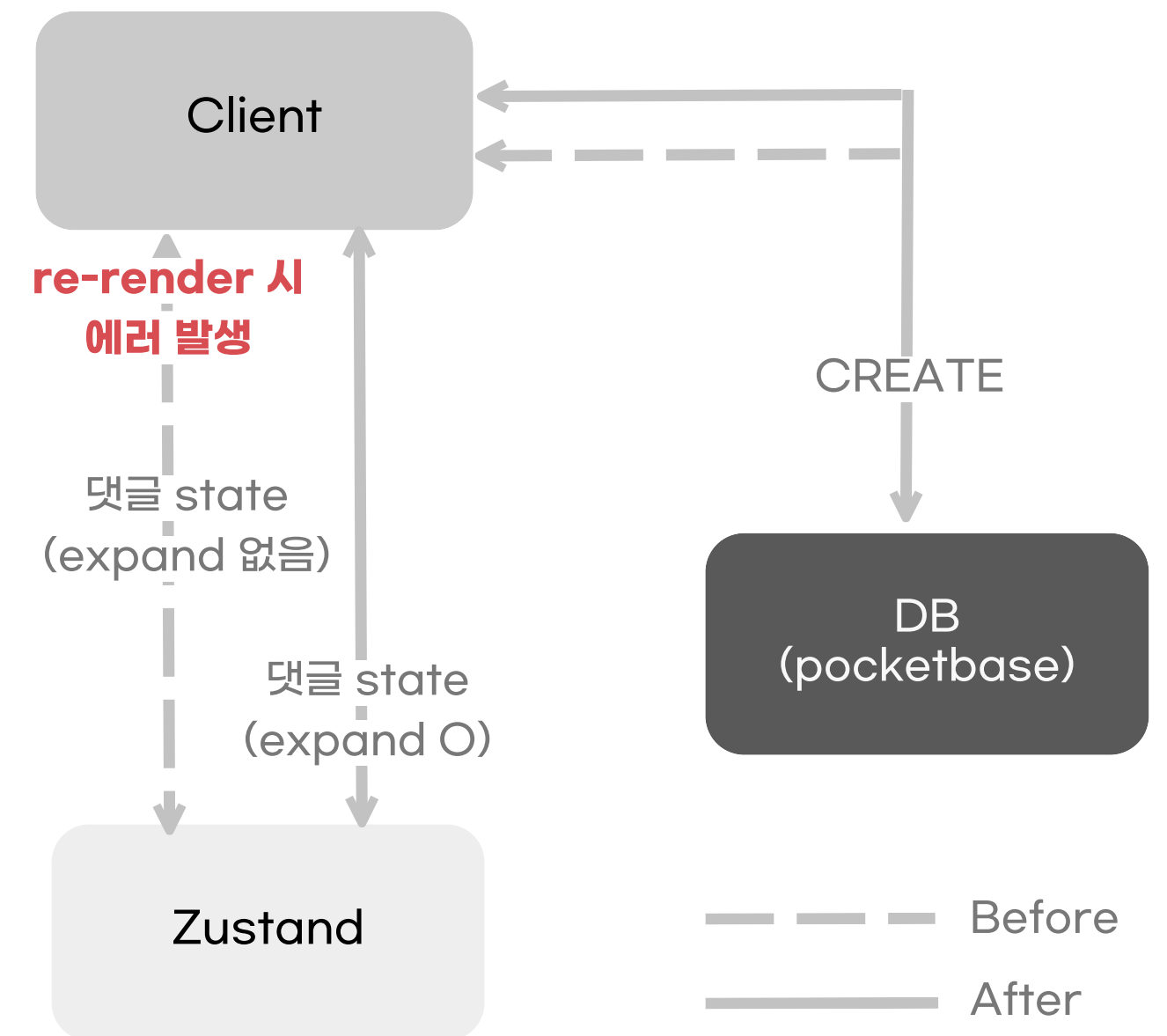
→ 새로 생성된 데이터에 expand가 없음

→ 이 데이터로 상태 업데이트됨

→ 리렌더링을 할 때 replies에 expand가 없어서 TypeError 발생

해결방법

- 데이터 create시 expand를 전달하도록 변경



사용자 경험을 위해 한 노력

성능

- svg sprite 사용
- CDN link font: @font-face 형식은 서버에 존재하는 폰트이기 때문에 성능 저하의 원인이 되어 pretendard와 같이 CDN을 지원하는 폰트는 CDN 방식으로 폰트를 설정
- Lazy Loading: 특정 페이지 방문 시에만 렌더링하도록 하여 성능 개선

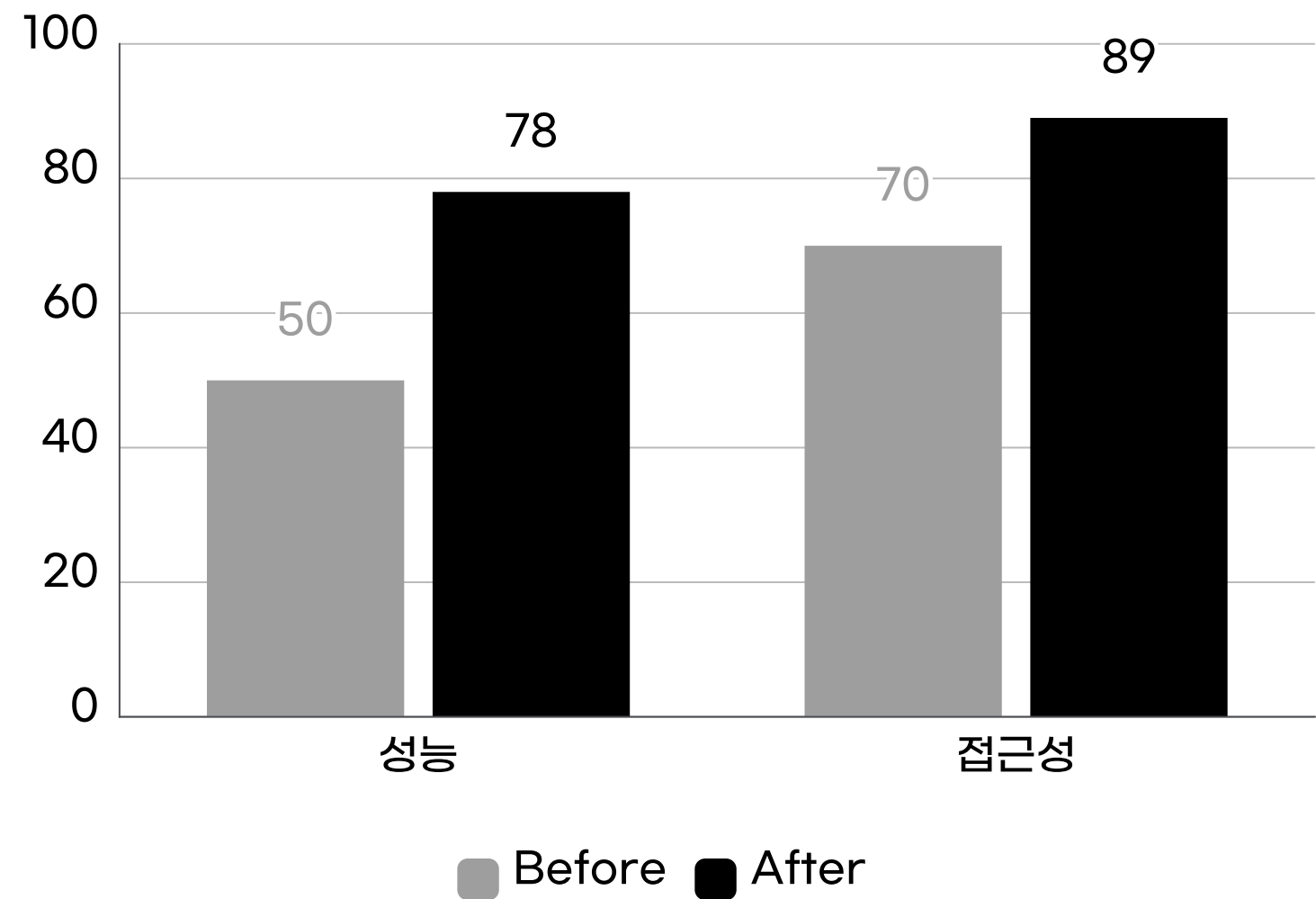
접근성

- symentic 마크업 준수
- 버튼 aria-label 추가
- HTML의 title 속성 사용: 버튼의 title을 추가하여 아이콘의 의미를 알 수 있도록 함

사용자 경험 개선

- favicon link: 다양한 브라우저, 플랫폼에서의 호환성을 위하여 다양한 용도의 favicon을 설정
- react-spinners: 비동기 작업(예: 데이터 로드, API 호출 등)이 진행 중일 때 아무런 피드백 없이 빈 화면이 나타날 경우 사용자 혼란을 완화하기 위하여 React-spinner를 설치하여 로딩 스피너를 추가함

Lighthouse 점수



성능 **56.00%** 
접근성 **27.14%** 

회고 및 프로젝트 성과

회고

아쉬웠던 점

- 시간이 부족하다 보니 코드 가독성을 고려하지 못함
- 캐싱에 대한 부분을 간과하여 잦은 fetch가 일어남 추후, react-query와 같은 비동기 데이터 관리 라이브러리 학습을 통해 보완하고 싶음
- 디자인할 시간이 부족하여 반응형으로 디자인하지 못함
- 데이터 통신할 때의 보안에 대한 고려 X

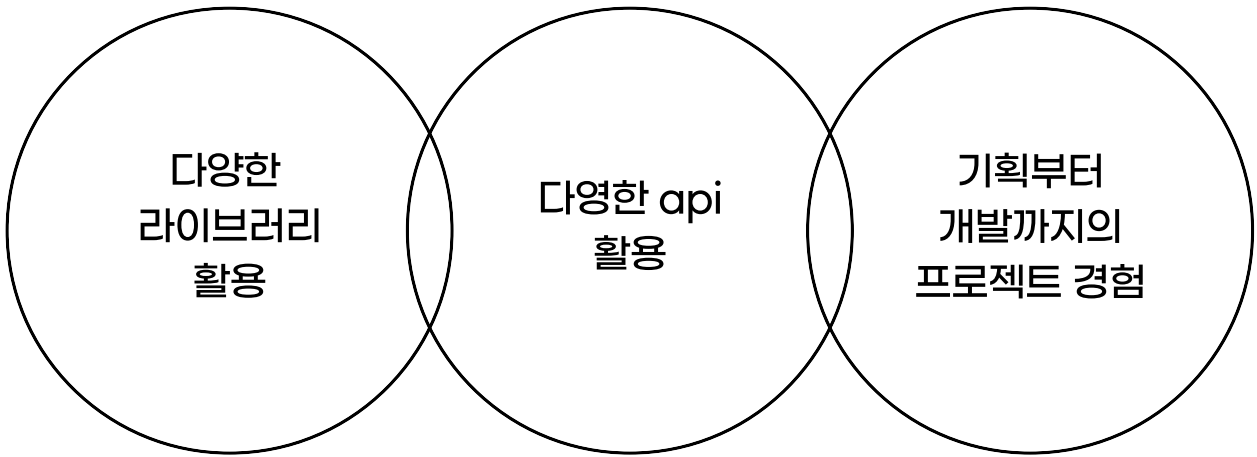
잘한 점

- 컴포넌트 index 파일을 통한 export, import로 코드 길이를 줄임
- zustand 상태관리를 통한 프롭스 드릴링 방지 -> 코드 간결화

프로젝트를 통해 배운 점

- 개발자에게 필요한 figma 사용 방법을 익힘
- 원하는 목표 달성을 위한 팀원들과의 의견 조율의 중요성
- SPA의 장점과 단점

프로젝트 성과



'새로운 기술 스택의 학습 및 적용'

개선할 점

- 회원 탈퇴 기능
- 주소 변경 기능
- 지도 위치 추적
- 카테고리 변경
- 잦은 데이터 fetch -> 캐시 적용